

Resume Screener

Project Documentation

An AI-powered resume-screening platform — FastAPI · React + TypeScript · PostgreSQL · MongoDB · SQLAlchemy · JWT/RBAC · pytest

Runs **fully offline with zero setup** (SQLite + a built-in mock LLM), or in a **production configuration** (PostgreSQL + MongoDB + a real LLM) — switched entirely through environment variables, with no code changes.

1. What it does

A recruiter creates a **job** (pasting the description); the app extracts its required skills. They then **upload resumes** (PDF / DOCX / TXT). For each resume the system:

- **Extracts the text** from the file.
- **Structures it** into a validated profile (name, email, skills, experience, education) using an LLM with a schema-validation retry loop.
- **Scores the fit** by blending embedding similarity with an LLM's holistic judgement — a 0–100 score plus a short reasoning string.
- **Ranks** all candidates into a shortlist, shows each candidate's matched vs. missing required skills, and moves them through a hiring pipeline (New → Screened → Interview → Offer → Rejected).

Everything is visible in a **React dashboard** — KPIs, fit distribution, a searchable talent pool, and a Kanban pipeline board.

2. Features

Screening & AI

- Resume parsing from PDF, DOCX, TXT
- Structured extraction into a strict schema, with automatic retry on validation failure
- Hybrid scoring = embedding similarity + LLM judgement, with explainable reasoning
- Skill-gap matching — matched vs. missing against the job's required skills
- Tunable scoring weights (live re-ranking), sort/filter, CSV export

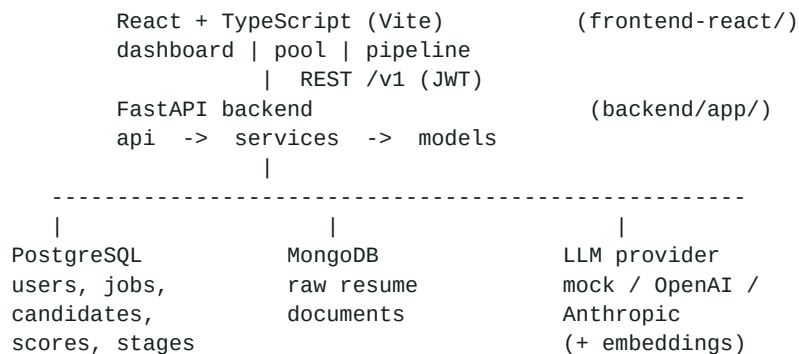
Workflow

- Hiring pipeline stages with a Kanban board
- Talent pool — searchable list of every candidate across jobs (server-side search)
- Role-aware dashboards — admins get an org-wide team view; recruiters get their own
- Job CRUD and a résumé document viewer

Engineering

- JWT auth + role-based access control (owner-scoped queries)
- Polyglot persistence — PostgreSQL (relational core) + MongoDB (résumé documents)
- Pluggable LLM provider (mock / OpenAI / Anthropic) behind one interface
- Alembic migrations, 46 automated tests (pytest)
- Decoupled React + TypeScript SPA over a REST API

3. Architecture



Layered backend: api/ (HTTP routes) → services/ (extraction, scoring, embeddings, cache, document store, LLM) → models/ (SQLAlchemy) + schemas/ (Pydantic).

Polyglot persistence: relational, transactional data (users, jobs, candidates, scores, pipeline stages, RBAC) lives in PostgreSQL; unstructured résumé artifacts (raw text + the full LLM extraction JSON) live in MongoDB, where a flexible document model fits naturally. Each has a graceful fallback (SQLite for the DB; the relational text column when Mongo is disabled).

4. How the AI works

Structured extraction (the core pattern)

LLMs don't reliably emit schema-perfect JSON, so extraction is a validate-and-retry loop:

```
LLM call -> parse JSON -> validate against a strict Pydantic schema
-> on failure, feed the validation error back into the prompt and retry
```

This makes extraction robust instead of hoping the model behaves.

Hybrid scoring

```
final = SIMILARITY_WEIGHT * (cosine_similarity * 100) + LLM_WEIGHT * llm_score
```

- **similarity** — cosine similarity between the job description and the résumé (embeddings)
- **llm_score** — an LLM's 0–100 holistic judgement + a short reasoning string
- Results are cached per (job_id, candidate_id) to avoid recomputation.

Skill-gap matching

Required skills (extracted from the job) are matched case-insensitively against each candidate's extracted skills to produce matched / missing lists and a coverage ratio.

Offline defaults: the LLM is a deterministic mock and embeddings use a lightweight hashing trick, so the whole pipeline runs with no API keys. Set a real provider via env to use actual models — the abstraction makes it a drop-in.

5. Tech stack

Layer	Technology
Frontend	React 18, TypeScript, Vite
Backend	FastAPI, Python 3.12
Relational DB	PostgreSQL (SQLite for local) via SQLAlchemy 2 + Alembic
Document DB	MongoDB via PyMongo (raw résumé artifacts)
Auth	JWT (PyJWT) + bcrypt + role-based access control
AI	Pluggable LLM (mock / OpenAI / Anthropic); offline embeddings
Parsing	pdfplumber (PDF), python-docx (DOCX)
Testing	pytest, mongomock — 46 tests

6. Getting started

Option A — Fully offline, zero setup (SQLite + mock LLM)

```
python3 -m venv .venv
.venv/bin/pip install -r backend/requirements.txt
cp backend/.env.example backend/.env          # DATABASE_URL=sqlite..., LLM_PROVIDER=mock
cd frontend-react && npm install && npm run build && cd ..
cd backend && ../.venv/bin/uvicorn app.main:app --reload --port 8001
```

Open <http://localhost:8001> and register an account. (Or use the Makefile: `make install`, `make web-build`, `make run`.)

Option B — Full stack (PostgreSQL + MongoDB)

```
brew services start postgresql@14
brew services start mongodb-community
createdb resume_screener
# backend/.env:
#   DATABASE_URL=postgresql+psycopg2://USER@localhost:5432/resume_screener
#   MONGODB_URL=mongodb://localhost:27017
cd backend && ../.venv/bin/alembic upgrade head
../.venv/bin/uvicorn app.main:app --reload --port 8001
```

Frontend development (hot reload)

```
cd frontend-react && npm run dev          # Vite on :5173, proxies /v1 -> :8001
```

7. Configuration

All settings are environment-driven (backend/app/core/config.py). Copy backend/.env.example → backend/.env and adjust.

Variable	Default	Purpose
DATABASE_URL	sqlite:///...	Relational DB; use postgresql+psycopg2:// in prod
MONGODB_URL	(unset)	Enables the MongoDB résumé document store
LLM_PROVIDER	mock	mock / openai / anthropic
OPENAI_API_KEY / ANTHROPIC_API_KEY	(unset)	Key for the chosen real provider
JWT_SECRET	change-me...	Token signing secret (set a strong value)
SIMILARITY_WEIGHT / LLM_WEIGHT	0.4 / 0.6	Hybrid-scoring blend
CORS_ORIGINS	["*"]	Allowed frontend origins

8. API reference

Interactive docs (Swagger) live at /docs when the server is running. Key endpoints:

Method	Path	Description
POST	/v1/auth/register	Create an account (role: recruiter/admin)
POST	/v1/auth/login	Get a JWT
GET	/v1/dashboard	Aggregated metrics (role-aware: team vs personal)
POST	/v1/jobs	Create a job (auto-extracts required skills)
DELETE	/v1/jobs/{id}	Delete a job (cascades candidates + docs)
POST	/v1/jobs/{id}/resumes/upload	Upload & screen resumes
GET	/v1/jobs/{id}/ranked-candidates	Ranked shortlist with scores
GET	/v1/jobs/{id}/skill-gap	Per-candidate matched/missing skills
GET	/v1/candidates	Talent pool across jobs (?search=)
PATCH	/v1/candidates/{id}/status	Move a candidate through the pipeline
GET	/v1/candidates/{id}/document	Raw résumé document (from MongoDB)

9. Roles & access control

- **Registration** picks a role: recruiter or admin. The role is embedded in the JWT.

- **Recruiter** — sees and manages only their own jobs and candidates (owner-scoped); a personal dashboard.
- **Admin** — an org-wide 'team' dashboard aggregating every recruiter's data, plus candidate-deletion rights.
- Every data query is scoped by role in the API — enforced server-side, not just hidden in the UI.

10. Testing

```
cd backend && ../../.venv/bin/pytest      # 46 tests
```

Tests run fully offline: SQLite for the DB and mongomock (in-memory MongoDB) for the document store — no servers or keys required. Coverage includes auth/RBAC, upload & extraction, scoring, skill-gap, talent pool, pipeline status, job deletion + cascade, and the MongoDB document store.

11. Project structure

resume-screener/	
- backend/	FastAPI application
- app/	
- api/	HTTP routes (auth, jobs, resumes, candidates, dashboard)
- services/	extraction, scoring, embeddings, cache, documents, llm/
- models/	SQLAlchemy models (Postgres/SQLite)
- schemas/	Pydantic request/response schemas
- core/	config + security (JWT/bcrypt)
- db/	engine/session, MongoDB, base
- alembic/	database migrations
- tests/	pytest suite
- frontend-react/	React + TypeScript (Vite) SPA
- frontend/	legacy single-file UI (fallback)
- sample_data/	sample resumes + job description
- Makefile	install / run / test / web-build helpers

12. Deployment

-
- **Frontend** → **Vercel** (perfect for the Vite SPA).
 - **Backend** → **Render / Railway / Fly** — a normal long-lived server; simpler than serverless for this stateful app. See backend/.env.production.example.
 - **Databases:** hosted PostgreSQL (Neon / Supabase) + MongoDB Atlas — free tiers. Switching is just the DATABASE_URL / MONGODB_URL env vars + alembic upgrade head.

13. Design decisions

-
- **Offline-first defaults** — clone and run in seconds; the same code scales to Postgres + Redis + a real LLM via env.
 - **Validate-and-retry extraction** — treats the LLM as unreliable and enforces a schema.

- **Hybrid, explainable scoring** — a cheap deterministic signal + an LLM judgement, always with a human-readable reason.
- **Polyglot persistence, justified** — relational data stays relational; résumés (which are documents) live in MongoDB.
- **Server-side authorization** — RBAC enforced in the API layer, not just the UI.

Built as a full-stack portfolio project demonstrating AI application/backend engineering, polyglot persistence, and a modern React frontend.